



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJE Y COMPILADORES (2026-I)

TEMA 4 - ANÁLISIS LÉXICO

**Diseño e Implementación de Analizadores Léxicos: Desde la Teoría de Autómatas
hasta la práctica con Metacompiladores**

Autores:

Adrian Reina (C.I. 31.317.970)
Fernando Centeno (C.I. 30.810.484)
Juan Longart (C.I. 31.882.343)
Rafael Rodriguez (C.I. 31.882.367)

Profesor:

Msc. Félix Márquez
fmarquez@e.uneg.edu.ve

Ciudad Guayana, Julio de 2026

ÍNDICE GENERAL

Resumen	3
Introducción	4
1. Actividad 1: Autómatas de Pila (PDA)	5
2. Actividad 2: Analizador Léxico para Archivos Docker (Python)	7
3. Actividad 3: Subconjunto de Lenguaje Rust (MiniRust) usando Flex	9
4. Actividad 4: Indagación de Flex en el Área de Seguridad Informática	11
Conclusiones	12
Referencias Bibliográficas	13

Resumen

El presente informe de investigación tiene como propósito analizar la fase inicial y primordial de la arquitectura de compiladores: el análisis léxico. Se examina la convergencia entre la teoría formal de máquinas abstractas y la ingeniería de software aplicada. Para establecer una base teórica sólida, se estudia el modelo de Autómatas de Pila (PDA), destacando su superioridad computacional sobre los Autómatas Finitos Determinísticos (AFD) al permitir el reconocimiento de lenguajes libres de contexto mediante memorias estructuradas LIFO.

Prácticamente, el documento contrasta dos filosofías de diseño de software. En primer lugar, se implementa una arquitectura manual "desde cero" utilizando el lenguaje Python para tokenizar archivos Dockerfile, demostrando eficiencia algorítmica $O(n)$ mediante expresiones regulares y control estricto de errores léxicos. En segundo lugar, se emplea el metacompilador Flex para generar automáticamente un autómata en lenguaje C destinado a procesar *MiniRust*, un subconjunto léxico del lenguaje de sistemas Rust.

Finalmente, se evalúa empíricamente la importancia crítica de los analizadores léxicos generados en C dentro de la seguridad informática contemporánea. Los hallazgos evidencian que infraestructuras de prevención de intrusos (IDS/IPS), analizadores de malware forense (YARA) y firewalls web (WAF) dependen de estas abstracciones formales para lograr inspecciones a velocidades de red lineales, mitigando ataques sin comprometer la latencia operativa del sistema.

Introducción

El análisis léxico representa la primera fase y el punto de entrada fundamental dentro de la arquitectura de un compilador moderno. En esta etapa, el código fuente —concebido inicialmente por el computador como una secuencia continua y desestructurada de caracteres (bytes)— es leído, verificado y agrupado en unidades lógicas con significado semántico denominadas **componentes léxicos** o **tokens**. A este proceso algorítmico se le conoce formalmente como **tokenización**.

El desarrollo del analizador léxico (también denominado *Lexer* o *Scanner*) se sustenta directamente en la teoría de máquinas abstractas: específicamente los Autómatas Finitos Determinísticos (AFD) y No Determinísticos (AFND). Estos autómatas poseen la capacidad matemática de reconocer los Lenguajes Regulares, los cuales ocupan el nivel Tipo 3 en la jerarquía de Chomsky y se definen programáticamente mediante Expresiones Regulares (Regex).

Para materializar esta fase en la construcción real de un compilador, la ingeniería ofrece dos grandes enfoques metodológicos que se exploran y desarrollan exhaustivamente en este informe:

- **Implementación Manual ("Desde Cero"):** Consiste en construir directamente el motor de lectura e iteración en un lenguaje de propósito general (como Python o C). Esto implica aplicar expresiones regulares complejas o implementar explícitamente la tabla de transiciones de estados del AFD en bucles condicionales, lo cual otorga al desarrollador un control absoluto sobre el manejo granular de errores.
- **Implementación mediante Metacompiladores (Flex):** Consiste en emplear herramientas generadoras especializadas que reciben como entrada un archivo de especificación declarativa. Estas herramientas procesan los patrones y compilan automáticamente el código fuente en C del autómata determinístico subyacente (*lex.yy.c*), acelerando el ciclo de desarrollo y minimizando la tasa de errores humanos en gramáticas extensas e industriales.

1. Actividad 1: Autómatas de Pila (PDA)

1.1 Definición Formal y Estructural

Un Autómata de Pila (*Pushdown Automaton* o PDA) es un modelo matemático de computación que extiende la funcionalidad estructural de un Autómata Finito mediante la adición de una memoria externa de acceso LIFO (*Last-In, First-Out*). Esta memoria infinita le otorga la capacidad de almacenar símbolos intermedios para realizar conteos, ejecutar verificaciones de balanceo posicional y recordar dependencias pasadas. Como resultado, el PDA está facultado para reconocer Lenguajes Libres de Contexto (Chomsky Tipo 2).

Formalmente, un Autómata de Pila se define como una tupla compuesta por 7 componentes rigurosos:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- **Q**: Conjunto finito de estados del control finito.
- **Σ** : Alfabeto finito de entrada (los símbolos que se leen de la cadena).
- **Γ** : Alfabeto finito de la pila (los símbolos que pueden apilarse o desapilarse).
- **δ** : Función de transición del autómata.
- **q_0** : Estado inicial del procesamiento.
- **Z_0** : Símbolo inicial que descansa en el fondo de la pila.
- **F**: Conjunto de estados de aceptación finales.

1.2 Supremacía y Ejemplos de Resolución

Para ilustrar la necesidad algorítmica del PDA, se toma como ejemplo el lenguaje de cadenas balanceadas: $L_1 = \{a^n b^n \mid n \geq 1\}$. Este lenguaje es matemáticamente imposible de reconocer por un AFD debido a que carece de memoria estructurada para "contar" cuántas letras 'a' han ingresado antes de validarlas contra las letras 'b'. El Autómata de Pila resuelve este teorema apilando el símbolo 'A' por cada 'a' detectada, y ejecutando una instrucción de desapilado por cada 'b' subsiguiente. Si al finalizar la lectura de la cadena la pila solo contiene a Z_0 , la cadena se acepta en un estado final garantizando un emparejamiento exacto.

1.3 Análisis Comparativo en la Jerarquía de Chomsky

La siguiente matriz descriptiva contrasta las limitantes y capacidades de las máquinas abstractas en el diseño de un compilador:

Criterio Evaluado	Autómata Finito (AFD / AFND)	Autómata de Pila (PDA)
Jerarquía de Chomsky	Tipo 3: Lenguajes Regulares	Tipo 2: Lenguajes Libres de Contexto (GLC)
Estructura de Memoria	Memoria finita y fija. Solo puede recordar su estado actual en Q.	Memoria infinita y estructurada gracias a la pila LIFO acoplada a los estados.
Capacidad de Conteo	Nula. No puede procesar anidamientos estructurales de forma dinámica.	Plena. Puede verificar llaves anidadas, paréntesis, bloques y emparejamientos exactos.
Rol en Compiladores	Analizador Léxico (Tokenización lineal).	Analizador Sintáctico (Validación de árboles lógicos).

2. Actividad 2: Analizador Léxico para Archivos Docker (Python)

Para la segunda actividad, el equipo diseñó y programó un lexer desde cero codificado en Python. El objetivo es procesar la sintaxis estricta de archivos Dockerfile. Este script ejecuta un análisis estático de texto plano (sin invocar al daemon de Docker), operando exclusivamente mediante patrones formales.

2.1 Arquitectura Lexicológica y Geometría de Patrones

El analizador define una lista ordenada de tuplas (DOCKER_TOKENS) para establecer la precedencia lógica y evitar colisiones léxicas. Entre las estructuras se identifican:

- **Directivas Reservadas (DIRECTIVE):** FROM, RUN, CMD, COPY, ENV, EXPOSE, WORKDIR.
- **Banderas de Ejecución (FLAG):** Cadenas que inician con doble guion (ej. `--platform=linux/amd64`).
- **Variables de Entorno (VARIABLE):** Capturas que obedecen al formato `$VAR` o expansiones `${VAR}`.
- **Regla General de Errores (MISMATCH):** Un carácter comodín al final de la matriz léxica que captura cualquier byte ilegal y genera un error sintáctico indicando la posición de falla.

La optimización central radica en el uso de grupos nombrados combinados con `re.finditer()`, asegurando una pasada algorítmica de complejidad computacional $O(n)$:

```
token_regex = '|'.join(f'(?P<{name}>{pattern})' for name, pattern
in DOCKER_TOKENS)
for mo in re.finditer(token_regex, input_text, re.MULTILINE):
    kind = mo.lastgroup
    value = mo.group(kind)
    if kind == 'MISMATCH':
        # Captura y aborta el flujo con error posicional
        raise RuntimeError(f'{value!r} unexpected on line
{line_num}')
```

2.2 Evidencia de Pruebas y Tolerancia a Fallos

Al ejecutar el analizador (`run_dockerfile_lexer.py`) sobre los escenarios de prueba proporcionados en la asignación, el sistema demostró la rigurosidad de las expresiones matemáticas subyacentes:

1. **Ejemplo 1 (Imagen Node.js - Válido):** El analizador identificó de forma exitosa los 36 componentes léxicos. Omitió correctamente los comentarios en línea e ignoró las tabulaciones profundas sin romper el control de flujo.
2. **Ejemplo 2 (Multi-stage build - Válido):** La arquitectura toleró directivas complejas de múltiples etapas extrayendo banderas específicas de propiedad, generando 65 tokens clasificados correctamente en consola.
3. **Ejemplo 3 (Inserción Ilegal - Rechazado Inmediatamente):** Al someter al lexer a una cadena contaminada con el carácter `¿`, la máquina rechazó compilar el archivo, abortando el proceso para evitar corrupciones en capas inferiores de un teórico *Parser*.

```
[!] ERROR LÉXICO en línea 7, columna 28:
```

```
Carácter o secuencia no reconocida: '¿'
```

```
Línea completa: >> RUN pip install -r requirements.txt ¿¿error_aqui?? <<
```

```
--- ESTADO: RECHAZADO (Se encontraron errores léxicos) ---
```


3. Actividad 3: Subconjunto de Lenguaje Rust (MiniRust) usando Flex

La construcción manual es efectiva para scripts estáticos, pero insostenible para crear los lenguajes de programación modernos. En esta actividad se aplicó la automatización industrial a través de **Flex (Fast Lexical Generator)**. El objetivo consistió en modelar un subconjunto de operaciones y tipados del lenguaje de sistemas Rust.

3.1 Especificación Léxica de MiniRust

El diseño lógico del lenguaje MiniRust abordó los siguientes conjuntos formales:

- **Keywords (Palabras Clave):** Declaraciones inmutables y mutables (let, mut), funciones y retornos (fn, return), directivas de control y bucles (if, else, while, for, in), booleanos (true, false) y la macro de impresión estándar (println!).
- **Taxonomía de Tipos:** Integración de tipos numéricos y alfanuméricos estáticos (i32, f64, bool, char, String).
- **Operaciones Compuestas:** Operadores relacionales de dos bytes evaluados mediante precedencia estricta (==, !=, <=, >=, ->, +=).

3.2 Anatomía del Metacompilador y la Regla Longest Match

El archivo de especificación rust_lexer.l se encuentra dividido en las tres áreas funcionales mandatarias del estándar de Flex:

1. **Sección de Definiciones (%{ ... %}):** Bloque nativo en lenguaje C. Define directivas globales y habilita la macro %option yylineno, crucial para incrustar el conteo de líneas de manera automática en el autómata subyacente.
2. **Sección de Reglas (%% ... %%):** Mapea las expresiones regulares al código de ejecución. Flex utiliza la heurística *Longest Match*: al encontrar un conflicto gramatical, la máquina de estados prioriza el patrón que consuma la mayor cantidad de caracteres del flujo de entrada continuo. En esta fase interviene yytext, un puntero automático de C que captura el lexema idéntico extraído.
3. **Sección de Procedimiento Principal:** Implementa la función autogenerada yylex() que arranca el escaneo binario.

3.3 Procedimiento de Compilación Estándar

La compilación expuesta en el entorno de pruebas requirió el uso de las cadenas de herramientas de sistema GNU (flex, gcc):

```
# 1. Generación del analizador C (lex.yy.c) desde las reglas léxicas
```

```
$ flex rust_lexer.l
```

```
# 2. Compilación del autómata a binario ejecutable
```

```
$ gcc lex.yy.c -o analizador_rust
```

```
# 3. Análisis sintáctico del código fuente anfitrión
```

```
$ ./analizador_rust ejemplo_rust1.rs
```

El output comprobó que Flex resolvió eficientemente los alcances de bucles, declaraciones e identificadores para calcular un factorial simulado, emitiendo un flujo constante de tokens sin dependencias cíclicas destructivas.

4. Actividad 4: Indagación de Flex en el Área de Seguridad Informática

La aplicación de lexers trasciende las herramientas de compilación de código fuente. En la seguridad perimetral moderna, detener ataques sin añadir latencia de red es el principal desafío operativo. Los sistemas de prevención y contención globales delegan este análisis crítico a autómatas determinísticos programados mediante herramientas como Flex.

IDS / IPS (Prevención y Detección de Intrusos: Snort y Suricata)

Redes corporativas masivas atraviesan tráfico concurrentes en el orden de los 100 Gbps. Interpretar en tiempo real anomalías de cabeceras TCP/IP con lenguajes de alto nivel es inviable debido a los sobrecostos de memoria y bucles de interpretación lentos. Motores como Suricata tokenizan las cabeceras binarias y opciones de payload directamente con lexers nativos, garantizando una complejidad temporal lineal $O(n)$ por trama interceptada sin saturar la Unidad Central de Procesamiento.

Análisis de Malware Forense (Reglas YARA)

YARA es considerado el estándar de facto internacional para la detección de firmas maliciosas. El núcleo principal del software (*libyara*) está construido matemáticamente apoyándose en **Flex y Bison**. Esto faculta al sistema a compilar miles de reglas de analistas (condiciones como meta:, strings: y volcados hex) en autómatas de estado finito ubicados directamente en la memoria RAM, escaneando archivos binarios infectados sin consumir IO de disco repetitivo.

Firewalls de Aplicación Web (WAF: ModSecurity)

Los WAF inspeccionan el comportamiento sintáctico que entra hacia el servidor web antes de interactuar con la base de datos. Para repeler ataques de inyección de código (*SQL Injection*, *Cross-Site Scripting*, *Path Traversal*), los cortafuegos tokenizan las URLs y los payloads JSON/URL-encoded asumiendo la labor de un compilador de intercepción; detectan palabras clave maliciosas (ej. UNION SELECT, <script>) y abortan la conexión TCP devolviendo un error HTTP de forma instantánea y perimetral.

Conclusiones

A lo largo del desarrollo de las actividades correspondientes al Tema 4, el equipo ha extraído las siguientes síntesis técnicas sobre la arquitectura y utilidad de la tokenización:

1. **Purificación del Flujo Semántico:** La fase de Análisis Léxico asume la labor fundamental de filtrar caracteres prescindibles, comentarios y separadores inertes del código fuente. Esta "purificación previa" reduce severamente la entropía que debe procesar el analizador sintáctico posterior (Parser), estabilizando la complejidad general del diseño de compiladores de producción masiva.

2. **Balance entre Control y Automatización:** La metodología de codificación manual "desde cero" concede a los desarrolladores niveles extremos de granularidad para atrapar e informar errores de sintaxis a nivel de posición. En contraposición, los metacompiladores basados en Flex son el estándar industrial primario para generar autómatas C infalibles a gran escala, erradicando los "falsos positivos" y sobrecargas léxicas que plagan las matrices implementadas por humanos.

3. **Evolución Teórica desde los Autómatas:** El análisis comparativo revela que los Autómatas Finitos (AFD) son herramientas excepcionales y ágiles para las tareas matemáticas lineales de lectura posicional continua (Lexer). No obstante, es estrictamente necesaria la memoria LIFO estructurada subyacente en el Autómata de Pila (PDA) para facultar resoluciones computacionales que impliquen bucles anidados, condiciones relacionales recursivas o evaluación de jerarquías arborescentes de ejecución (Sintaxis Lógica).

Referencias Bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compiladores: Principios, técnicas y herramientas (El Libro del Dragón)*. 2da Edición. Pearson Educación.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación*. 3ra Edición. Addison-Wesley.
- Levine, J. (2009). *Flex & Bison: Text Processing Tools*. O'Reilly Media.
- Márquez, F. (2026). *Guía de Estudio: Tema 4 (Análisis Léxico)*. Universidad Nacional Experimental de Guayana (UNEG).
- Documentación Oficial del Proyecto YARA: The pattern matching swiss knife for malware researchers. <https://virustotal.github.io/yara/>
- Documentación y Especificación del Lenguaje Rust.
<https://doc.rust-lang.org/reference/>